

packages/graph-core/src/schema.ts

Kind: added · Baseline: 8e0dc031dcfc · Target: NovaDiff · Generated: 2026-05-19T19:42:08.647Z

Overview

A new `packages/graph-core/src/schema.ts` file was added (+701lines). It introduces Zod schemas for knowledge-graph entities and a four-tier validation pipeline.

The change adds an import (R1), an `EdgeTypeSchema` enum (R4), and several alias maps (R17 , R78 , R128 , R139). Functions `sanitizeGraph` (R148), `autoFixGraph` (R196), `normalizeGraph` (R467), and `validateGraph` (R504) implement sanitization, auto-correction, alias resolution, and validation.

Supporting helpers `buildInvalidCollectionIssue` (R452) and `buildErrors` (R461) aggregate diagnostics. Public interfaces `GraphIssue` (R436) and `ValidationResult` (R443) expose error information, and `KnowledgeGraphSchema` (R426) defines the full graph shape.

Key changes

- **Imports & enums** – `import { z } from "zod"` (R1) and `EdgeTypeSchema` with 35 edge types (R4).
- **Alias maps** – `NODE_TYPE_ALIASES` (R17), `EDGE_TYPE_ALIASES` (R78), `COMPLEXITY_ALIASES` (R128), `DIRECTION_ALIASES` (R139).
- **Sanitization** – `sanitizeGraph` normalizes nulls, lower-cases strings, and removes optional fields (R148).
- **Auto-fix** – `autoFixGraph` supplies defaults for missing `type`, `complexity`, `tags`, `summary`, and coerces edge weights (R196).
- **Normalization** – `normalizeGraph` replaces aliased node/edge types with canonical ones (R467).
- **Validation pipeline** – `validateGraph` orchestrates sanitization, normalization, auto-fix, collection checks, and schema validation for nodes, edges, layers, and tour steps (R504).
- **Error handling** – `buildInvalidCollectionIssue` and `buildErrors` aggregate fatal and non-fatal issues (R452 , R461).
- **Exported interfaces** – `GraphIssue`, `ValidationResult`, and the full `KnowledgeGraphSchema` are now public.

Impact

- **Correctness** – Every graph component is validated against Zod; invalid collections trigger fatal errors.
- **Observability** – The `issues` array provides fine-grained diagnostics for downstream consumers.
- **Maintainability** – Centralized schema definitions reduce duplication across the repo.
- **Performance** – Auto-fix and normalization run on every validation; may add overhead for large graphs.

- **Compatibility** – Existing code importing `schema.ts` must reference the new exports; no breaking API changes within the module itself.

Risks & follow-ups

- **Test coverage** – Ensure unit tests exercise all alias mappings and auto-fix scenarios.
- **Performance regression** – Benchmark `validateGraph` on large datasets to confirm acceptable latency.
- **Schema drift** – If other packages rely on older node/edge type definitions, update imports accordingly.
- **LLM integration** – Verify that LLM-generated graphs are correctly normalized; watch for edge cases where aliases conflict with canonical values.